02 — AI FOUNDATIONS

AI v/s ML v/s DL v/s Gen AI;

AI
↳ ML
↳ DL
↳ GEN AI
↳ LLMS.

AI [Artificial Intelligence]

AI is broad field of creating systems that performs the tasks that normally require human-like intelligence.
Eg: Reasoning, Planning, Perception, Decision-making, language understanding.

Machine learning

ML is a way of building AI systems where the model learns patterns from data rather than being explicitly programmed with every rule.
Traditional → Rules + Data ⇒ Output
ML ⇒ Data + Expected O/Ps → model → learned Rules

Deep learning

Uses neural networks with many layers to learn complex patterns. Used in: Computer Vision →; Speech Recognition, NLP, Recommendation systems, Gen AI.

Generative AI

creates new content. Eg: Text, Images, Audio, Video, Code.
An LLM is a type of Gen-AI model specialized primarily around language.

Model: mathematical system that has learned patterns from data and can use those patterns to make predictions or generate outputs.
Eg: Traditional Data → learning Algos → Model → New Input → Prediction.
Eg: Input: "House has 3 bedrooms, 2 baathrooms ..." → ML model
→ Predicted Price: ₹ 50 Lakh.

Training v/s Inference: Training: the model learns from data.
Data → Model → Adjust parameters →
→ Better Predictions.

Training requires : large datasets, significant compute, GPUs / TPUs, time.

Inference : the trained model is used to generate a prediction / output.

New input → Trained Model → output.

Example : when you send a prompt to an LLM, you're generally doing inference, not training the model from scratch.

Parameters : A model contains parameters that are adjusted during training.

Input → Parameters → output.

During Training : Prediction → Compare with expected answer → calculate error → Update parameters.

Modern neural networks can have millions, billions, or more parameters.

More parameters ≠ automatically better model.

Performance also depends on : Training data, Architecture, Training Process, Data Quality, Compute, Evaluation, Alignment / post training.

Dataset : information used to train / evaluate model.

⊙ For supervised learning, Input + label [eg: email → spam / not → Not Spam.

⊙ Dataset is commonly divided into :
→ Training set → Validation set → Test set.

⊙ Training set : used to learn parameters.

⊙ Validation " : " during development to tune choices.

⊙ Test set : " for final evaluation.

Overfitting v/s Underfitting

Overfitting : model learns the training data too specifically.

→ Training performance : Very Good

→ New Data " : Poor.

It has effectively learned patterns / noise specific to the training data instead of generalizing well.

Underfitting : models is too simple to capture important patterns.
Training performance → Poor.
New data " → Poor

Goal : Learn useful patterns
→ Generalise to unseen data.

Generalization
how well a model performs on data it hasn't seen during training. This is one of the most important ideas in ML.
you don't want : learn underlying patterns → Work on new data.

Supervised learning : the model learns from labeled data.
Input → known output eg : Hrs studied → Exam score.
Tasks :—
⊛ Classification : Predicts a category. Email → Spam / Not Spam.
⊛ Regression : Predicts a continous value. House features → Price.

Unsupervised learning : There are no predefined labels.
The model tries to discover patterns or structure.
eg : Customer data → Clustering → Customer groups. detection
Common eg : Clustering , Dimensionally reduction , Anomaly

Reinforcement learning : An agent learns through interaction with an environment.
Agent → Action → Environment → Reward / Penalty → learning.
Eg. Game Agent → Move → Game State → Reward.
The goal is to learn a policy that maximizes cumulative reward.

Neural Networks : A neural networks consists of interconnected computational units arranged in layers.
Input layer → Hidden layer → Hidden layer → output layer.
Each connection has a weight.
The network learns appropiate weights during training.

Activation Functions : Neural networks use activation functions to introduce non-linearity.
Common ones : ReLU , Sigmoid , Tanh , Softmax.
ReLU : $f(x) = \max(0, x)$ so $\Rightarrow$ $x < 0 \to 0$; $x > 0 \to x$
Softmax : Often used to convert scores into a probability distribution accross classes.

Examples: Cat → 0.70 Rabbit → 0.10.
Dog → 0.20

Loss function: The model needs a way to measure how wrong its prediction is. That's the purpose of a loss function :- Prediction → Compare with target → Loss.
Generally, lower loss ➡ better fit to the training objective.
Eg: Mean Squared Error, Cross Entropy loss.

Gradient Descent: The model wants to minimize its loss.
Start → calculate loss → Calculate gradients → Adjust Parameters →
→ calculate loss again → Repeat.

"Gradient descent is an optimization method used to adjust model parameters in a direction that reduces the loss."

Learning Rate: It controls how much parameters are changed during each optimization step.
⊙ Too large: May overshoot → unstable training
⊙ Too small: Training becomes very slow.
→ So you need appropiate learning rate.

Epoch, Batch & Iteration:
Epoch → one complete pass through the training dataset.
Batch → a smaller portion of the dataset processed at once.
Iteration → One parameter-update step for a batch.
Eg: Dataset = 10,000 samples Batch size = 100
1 epoch = 100 batches = approximately 100 iterations

NLP: Natural lang. Processing deals with computers processing human language.
Applications: Translation, Sentiment Analysis, Text classification, Question Answering, Summarization, Chatbots, Information extraction.
(#) Modern LLMs are a major development in NLP.

Tokens : LLMs generally don't process text as individual words in the simplistic sense.
Text is broken into tokens. eg: Artificial Intelligence.
↳ might be represented as multiple tokens depending on the tokenizer.

Text → Tokenizer → Tokens → Token IDs → Model

Important because :
→ context windows are measured in tokens.
→ Input/Output costs are often token-based
→ long prompts consume more context.

Embeddings : represents information as a numerical vector. "Python Programming" → [0.21, −0.42, 0.78, ...]
Semantically related things can have similar vector representations.
Used in : Semantic search ; Recommendation ; RAG ;
Clustering ; Similarity search.
important when you reach AI agents / AI applications.

Transformers : Modern LLMs are largely based on the transformer ~~activit~~ architecture.
Input tokens → Embeddings → tranformer layers → output probabilities → Generated Tokens
Transformers use attention mechanisms to model relationships between tokens.

Attention : allows model to determine which parts of the input are relevant to each other.
for eg: The student submitted her project because she finished it first".
The model needs "she" refers to. Attention helps the model represent relationships b/w tokens.
you don't need to derive attention equation yet.
know: "Attention allows a model to weigh the relevance of different tokens when processing a sequence".

LLM – Large Language Model

→ LLM is a large neural-network model trained on huge amounts of text/data to perform language-related tasks.
Example of capabilities: Generate text, Answer questions, Summarize, Translate, Generate Code, Extract information, Reason over provided context.

But remember: An LLM is not a database.
It generates outputs based on learned patterns & its current context.

How does an LLM Generate Text?

Prompt → Tokenization → Tokes → Model → Probability Distribution → Select next token → Add token to Content → Repeat.

For eg: "The capital of France is"
the model assigns probabilities to possible next tokens:-
Paris → high probability
London → lower "
Berlin → " "
] It then generates the next token and continues.

Hallucination

: An LLM can produce an answer that sounds convincing but is factually incorrect.
This is commonly called a hallucination.
Example:
- User → "Who wrote a fictional paper called X?"
- LLM → Provides a convincing-looking author, journal & year.

→ The information may not actually exist.
Therefore: Confident output ≠ Guaranted truth.
This is extremely important for AI systems.

Why Do LLMs Hallucinate?

→ There isn't one single cause.
→ Relevant factors include: Insufficient information, Ambiguous prompts, Model limitations, Outdated knowledge, poorly represented information, generation uncertainity.

lack of external verification.
This is why AI systems often need grounding, retrieval, validation, and evaluation.

Context Window

→ An LLM can only process a finite amount of context in one interaction.

System instructions → Conversation → Documents → User Prompt → Generated output.

must fit in within the model's available context limits.
larger context doesn't automatically mean the model will perfectly use every piece of information.

Temperature: controls randomness in token selection in systems that expose this parameter.

lower temp → more predictable output.
higher " → more variation / randomness.

For example: low temp → factual / structured tasks
higher " → creative generation.

Don't treat temperature as a direct 'intelligence' setting.

AI Limitations:

Models can →
- detect patterns
- generate content
- classify info
- Summarize
- Transform text
- Generate code
- Work with structured prompts / context

Model can struggle with:
- Factual accuracy
- Ambigous instructions
- Missing context
- long or complex reasoning
- Novel situations
- Reliable arithmetic without appropiate tools.
- Knowing whether their own claims are true.

AI output → Verification → Reliable application.
is often necessary.

AI + Tools : Modern AI systems don't have to rely solely on the model.
You can give AI system access to tools:

LLM → tool
↳ calculator, Search, DB, API, code execution

Eg: User: "What's today's weather?
LLM alone: may not know current weather.
LLM + weather API: retrieves current data + explains it.

This concept becomes crucial later for AI agents.

RAG – Retrieval-Augmented Generation

Instead of asking the LLM to answer purely from its internal learned knowledge: Question → LLM → Answer
RAG does: Question → Retrieve relevant information →
→ Give info + Question to LLM → Generate answer

Eg: Company docs → Embedding → Vector DB → Relevant docs → LLM → answer.

RAG can help ground responses in specific external information, though it doesn't guarantee correctness.

Fine-Tuning v/s RAG

- RAG: Give the model additional info at inference time.
Model stays mostly unchanged + external retrieved content.
- Fine tuning: Train/update a model using examples to adapt its behaviour or capabilities for a particular task.
Base Model → Training on task-specific examples → Adapted model.
- Need current/private/reference information?
→ RAG may be appropriate.

Need consistent task-specific behaviour/style?
→ Fine-tuning may be considered.
The actual choice depends on the application.

AI-Evaluation : An AI system shouldn't be judged by :
"The answer looks good"
measurable criteria needed.
For example : Accuracy, Relevance, Correctness, Completeness, Safety, Consistency, Latency, Cost.

Eg: Prompt → Model → output → Evaluation Criteria → Score/Pass-Fail

Most Important Mental Model

Data → Training → Model → Inference → output → Evaluation → Verification

And for modern AI Application ⇒

User → Prompt → LLM → Tools/Retrieval → LLM → Response → Evaluation/Guardrails.

X——X——X——X——X